PilotFish Directory / File Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as the **outbox folder** at the end of a route. When a transaction finishes processing, this transport writes the message body to a file on disk — choosing the directory, name, and what to do if that file already exists (append, overwrite, or create a new unique name).

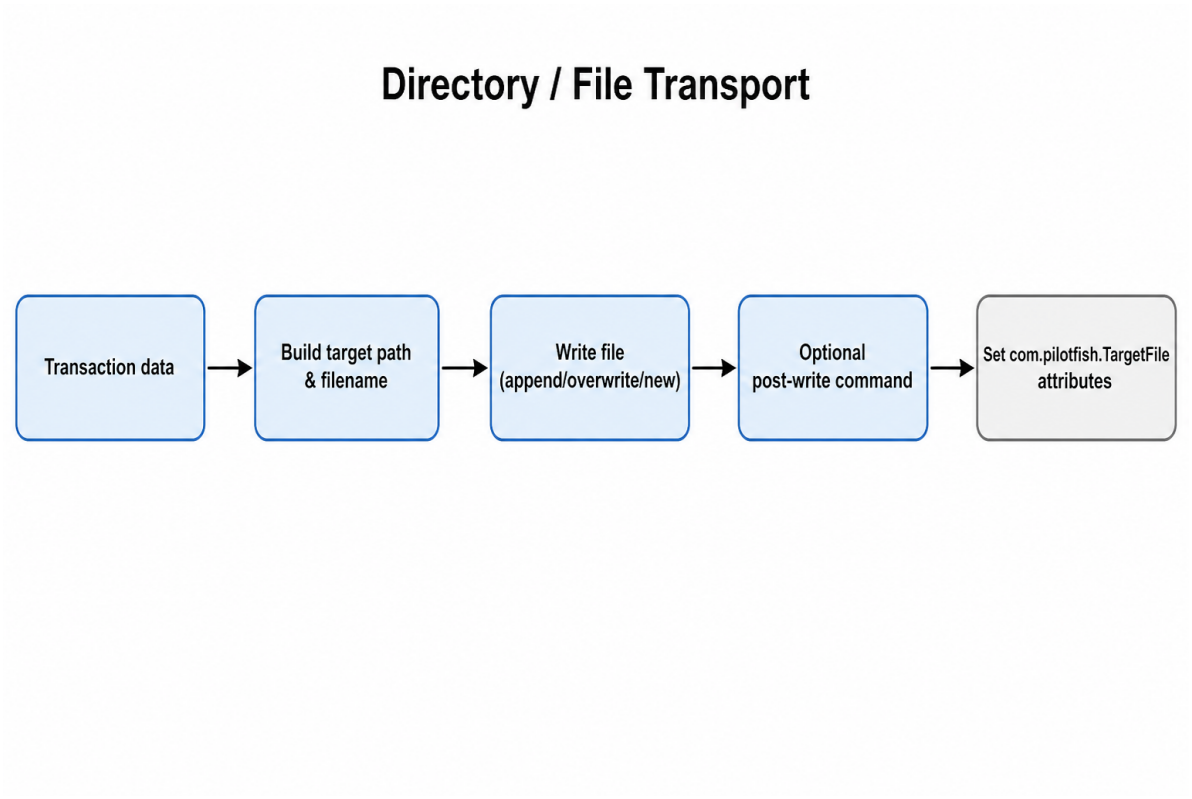

Figure 1: At a glance (plain English)

- **Writes** transaction payload bytes to a configured directory or full path.
- **Supports** dynamic file names using date/time tokens, transaction ID, and attributes.
- **Handles** name conflicts via append, overwrite, numeric suffix, or date-stamp pattern.
- **Optionally** runs a shell command after the file is written (batch-sensitive rename supported).

Purpose

This document is a **deep dive** into the **Directory / File** transport as shipped in `eip.war.hs.26R1.11`: how it resolves target paths, writes streams, manages file-exists behavior, batch-sensitive `.tmp` staging, post-write commands, and target-file transaction attributes.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-file-1.0.1.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.file.DirectoryTr`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../Di`
Transport type string	`Directory / File`
Description	`Stores data as files in a specified directory.`
Help ID	`console.transport.directory`
Category	`FILESYSTEM`

> Derived from WAR bytecode (CFR). Narrative follows executable logic in `executeTransport`.

---

1. Conceptual model

Directory / File is the standard **filesystem sink** transport. It copies the transaction `InputStream` to a `FileOutputStream`, optionally appending, and publishes where the file landed via attributes.

```
TransactionData.dataStream
        |
        v
 Resolve path (directory+name OR full path)
        |
        v
 Apply conflict / size-quota naming (Util.getNewFileName)
        |
        v
 mkdirs(parent) if needed
        |
        v
 setTargetFileTxAttributes(File)
        |
        v
 synchronized write (append or truncate)
        |
        v
 optional BatchSensitive .tmp -> final rename
        |
```

```
        v
optional Run Command (cmd.exe or shell -c)
```

It does **not** read files back into the transaction stream after write (stream is closed after copy).

---

2. High-level behavior (`executeTransport`)

From decompiled `DirectoryTransport.executeTransport`:

1. **Resolve target file path**
   - If **Specify full file path** = `Enabled` → use **Path to file** (`FullPathToFile`) as the complete target string.
   - Else build from **Target directory** + parsed **Target file name** + **Target file extension**.
2. **Parse / tokenize file name** via `parseFileName` → `createFileName` (section 4).
3. **Conflict handling** when **If file exists** = `Create New`:
   - Empty base name → prefix `DEFAULT_FILENAME`.
   - With **File Name Conflict Pattern** → append `SimpleDateFormat` suffix.
   - Without pattern → `Util.getNewFileName(name, directory)` (numeric suffix).
4. **Non-create-new, non-batch** → `directory + separator + fileName`.
5. **Batch Sensitive** → write to `.../fileName.tmp` (`.tmp` appended to full path string).
6. **Maximum File Size** > 0 → `Util.getNewFileName(basename, dir, maxFileSize)` rotates when quota exceeded.
7. **Create parent directories** if missing (`mkdirs`).
8. **Set attributes** `com.pilotfish.TargetFile`, `TargetFileName`, `TargetFileExtension`.
9. **Write** (synchronized on transport instance):
   - Append mode when **If file exists** = `Append` (or legacy `"true"`).
   - Else truncate/create.
   - `DataUtil.copy` from transaction stream; close streams.
10. **Batch Sensitive completion**: when `com.pilotfish.BatchSize` decrements to 0, rename `.tmp` by stripping first char and last 4 chars of path (legacy batch rename logic).
11. **Run Command** if non-blank: resolve tokens via `StringUtil.resolveCommand`, execute via `SystemCommandExecutor` (`cmd.exe /c` on Windows, `{Shell} -c` on Unix).

Errors → `EIPException("I/O error while performing transport: ...")`. Non-IO exceptions in the main try are logged as critical but not always rethrown.

---

3. File name token algorithm (`createFileName`)

Applied to the configured name (+ extension) before writing:

Token	Replacement
`{SourceFile}`	Name of `com.pilotfish.SourceFile` attribute if it is a `File`
`{DATE}`	`yyyy-MM-dd`
`{TIME}`	`HH-mm-ss`

Token	Replacement
`{FDATETIME pattern}`	`SimpleDateFormat(pattern)` — pattern between token and closing `}`
`{TrID}`	`data.getTransactionID()`
`{TrAttr name}`	Attribute value or empty string

Invalid `SimpleDateFormat` patterns in **File Name Conflict Pattern** throw `EIPException` with guidance to see Java docs.

---

4. Configuration reference

4.1 Module-specific options

UI label	Tag	Type	Default	Tab	Required	Nuances
Target directory	`TargetDirectory`	Path	`""`	Basic	Yes*	Visible when full path = Disabled
Target file name	`FileName`	String	`""`	Basic	Yes*	OGNL/enhanced supported via manager
Target file extension	`FileExtension`	String	`""`	Basic	No	Prepended with `.` when set
Specify full file path	`UseFullFilePath`	Check	`Disabled`	Basic	No	`Enabled` / `Disabled`; toggles dependent fields
Path to file	`FullPathToFile`	File	`""`	Basic	Yes*	Required when full path enabled
If file exists	`AppendToFile`	Choice	`Create New`	Basic	Yes	`Append`, `Create New`, `Overwrite`; legacy bool mapped
Maximum File Size	`MAXIMUM_MEMORY_SIZE`	Memory (bytes)	`-1`	Basic	No	`-1` = infinite; file may exceed limit before rotation

UI label	Tag	Type	Default	Tab	Required	Nuances
File Name Conflict Pattern	`FileNameConflictPattern`	String	`""`	Advanced	No	Only when Create New; Java `SimpleDateFormat`
Batch Sensitive	`BatchSensitive`	Boolean	`false`	Advanced	No	Uses `.tmp` + `com.pilotfish.BatchSize`
Run Command	`Command`	String	`""`	Advanced	No	Post-write; calls `data.getDataStream().res` first
UNIX/Linux Shell	`Shell`	String	`/bin/bash`	Advanced	No	Used for `-c` on non-Windows

* Required when the dependent panel is active (`isConfigurationComplete` via item rules).

Dependency panels

- `UseFullFilePath=Disabled` → directory, file name, extension.
- `UseFullFilePath=Enabled` → full path only.
- `AppendToFile=Create New` → conflict pattern field.
- `AppendToFile=Append` → maximum file size field.

4.2 Inherited AbstractTransport / AbstractEIPModule

`DirectoryTransport` does **not** add inherited conditional flags beyond standard module completeness checks. Transport lifecycle:

- `sendData` → `checkRequiredData` → `executeTransport` → `setTransactionStateComplete()` on success.
- Failure → `setTransactionStateError()` + `EIPException`.

`AbstractTransport.supportsBatchSynchronization()` returns `false` (batch coordination uses attribute + `.tmp` convention instead).

---

5. Transaction attributes

5.1 Written by this transport

Attribute key	Type	When set
`com.pilotfish.TargetFile`	`File`	Before write, absolute path
`com.pilotfish.TargetFileName`	`String`	Base name without extension
`com.pilotfish.TargetFileExtension`	`String`	Extension from final path

5.2 Consumed (optional)

Attribute key	Usage
`com.pilotfish.SourceFile`	`{SourceFile}` token substitution
`com.pilotfish.BatchSize`	`MutableInt`; batch-sensitive rename when reaches 0

---

6. Worked example — append with size rotation

Configuration:

- Target directory: `/data/out`
- File name: `claims_{DATE}.txt`
- If file exists: **Append**
- Maximum File Size: `10485760` (10 MB)

Behavior:

1. First transaction appends to `/data/out/claims_2026-08-04.txt`.
2. When file exceeds 10 MB, `Util.getNewFileName` picks the next rotated name before append continues.
3. Attributes reflect the actual file written on each transaction.

---

7. Known quirks and caveats

1. **Legacy Append values** — `AppendToFile` accepts old boolean strings `"true"`/`"false"` mapped to Append / Create New.
2. **Batch rename** — uses `substring(1, length-4)` on the `.tmp` path; assumes a leading character convention from batch listeners.
3. **Overwrite mode** — no explicit branch in decompile for `Overwrite`; non-append opens `FileOutputStream` without append flag (truncates).
4. **Stream consumed** — input stream closed after copy; downstream stages should not expect to re-read original stream.
5. **Run Command** requires reset-able stream on `data.getDataStream()`; may fail if stream is not reset-capable.
6. **Synchronized write** — instance-level lock serializes concurrent transports sharing the same module instance.
7. **Critical catch swallows** — generic `Exception` in outer try logs error without always propagating (I/O errors do throw).

---

8. Operational recommendations

Goal	Guidance
Idempotent drops	Use **Create New** + date pattern to avoid clobbering
High-volume append logs	Set **Maximum File Size** to rotate before filesystem limits
Downstream pickup	Publish `com.pilotfish.TargetFile` to trigger file watchers
Batch responses	Pair **Batch Sensitive** with batch-aware listeners setting `BatchSize`
Post-processing	Use **Run Command** for antivirus scan or `chmod`; test shell path on Linux

---

9. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
Directory / File transport	`com/pilotfish/eip/modules/file/DirectoryTransport.java`
Abstract transport base	`com/pilotfish/eip/extend/AbstractTransport.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-modules-file-1.0.1.jar`

---

10. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-modules-file-1.0.1.jar
  -> CFR decompile
  -> Documents/Transports/26R1.11/PilotFish-Directory-File-Transport-Reference-26R1.11.(md|pdf)
```

Deep-dive documentation from WAR decompile – Directory / File Transport – 26R1.11 – August 2026.